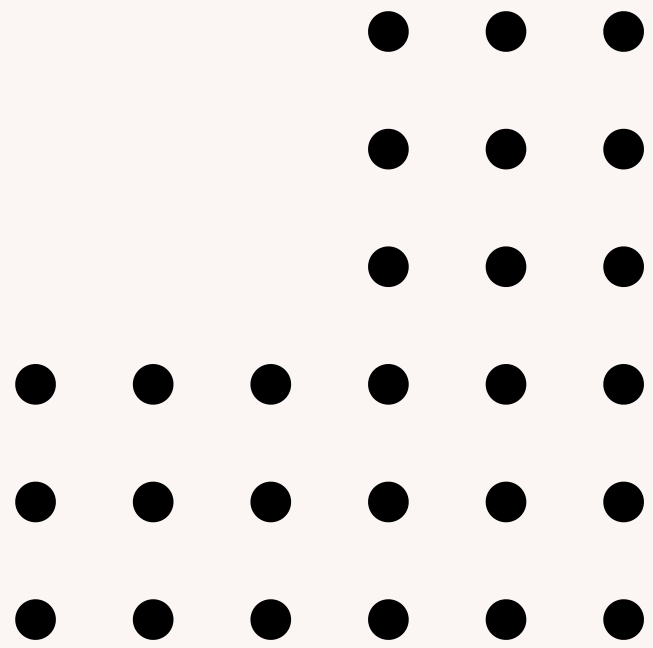
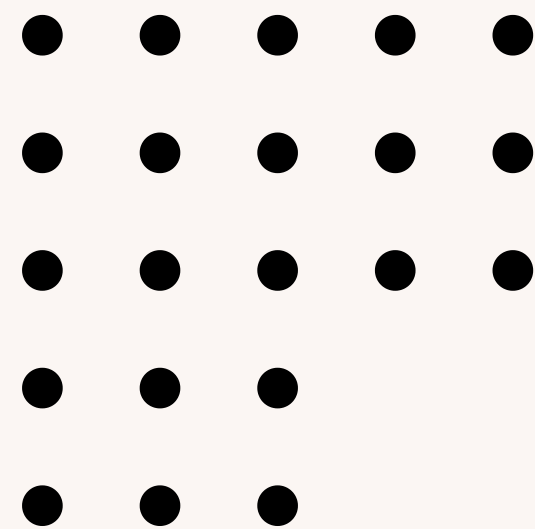


DESAFIO 1

Disciplina: GBC073 - Inteligência computacional

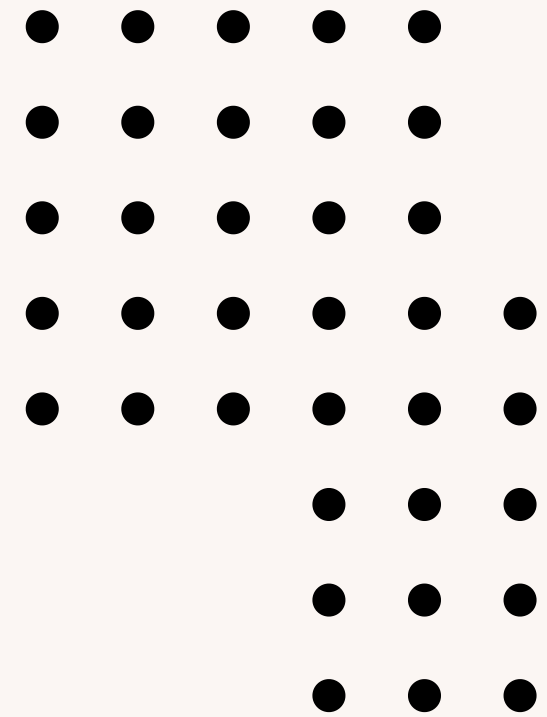
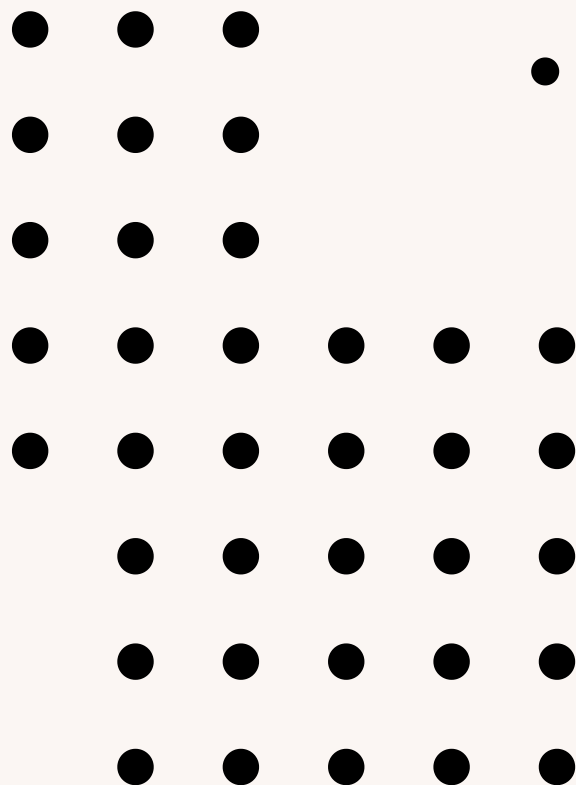
Nomes: Felipe Canabarro Graffunder

Isabela de Paula Barbosa



OBJETIVO

- O perceptron utiliza uma fronteira de decisão linear
- Alguns conjuntos de dados não são linearmente separáveis
- A solução consiste em transformar os dados de entrada
- **A transformação deve:**
 - Ser determinística
 - Não gerar valores NaN ou inf
 - Ser rápida
 - Utilizar no máximo 64 características
- **Ideal principal:**
 - Criar novas características para representar padrões não lineares



SOLUÇÃO INICIAL

A solução inicial original realizava apenas a padronização dos dados e o cálculo dos quadrados

```
class Submissao:
    DIM_MAX = DIM_MAX

    def fit(self, X):
        self.mu = X.mean(0)
        self.sd = X.std(0) + 1e-8

    def phi(self, X):
        Xs = (X - self.mu) / self.sd
        return torch.cat([Xs, Xs ** 2], dim=1)
```

Limitações

Essa transformação não representava bem padrões como:

- XOR
- Duas luas
- Círculos
- Espirais
- Esfera em 10 dimensões

PADRONIZAÇÃO DOS DADOS

- A primeira etapa solução foi padronizar os dados:

`self.mu = X.mean(0)`

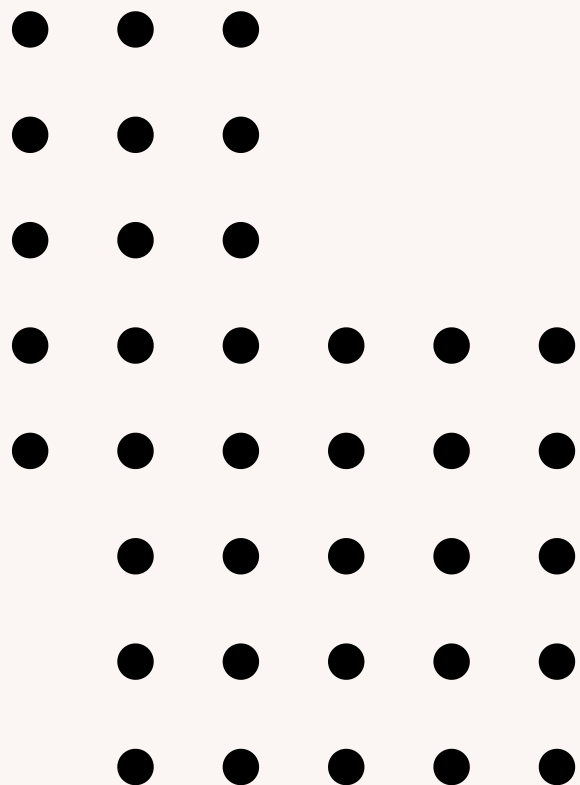
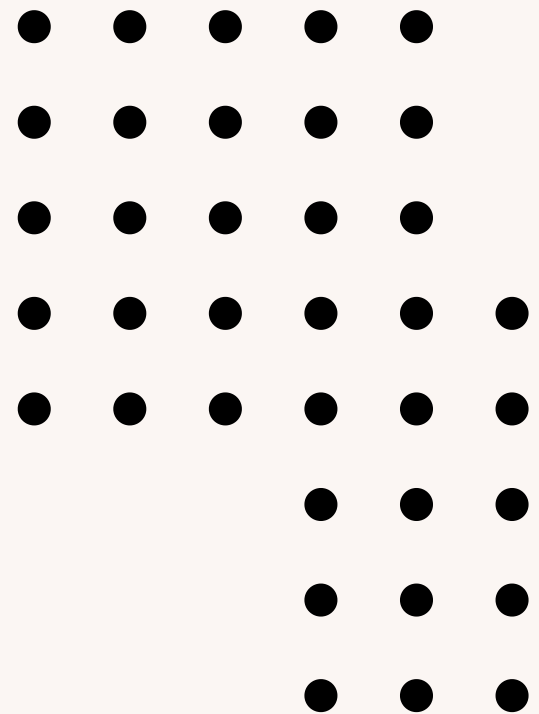
`self.sd = X.std(0) + 1e-8`

- Na transformação:

`Xs = (X - self.mu) / self.sd`

- Por que padronizar?

- Evita que uma variável domine as outras
- Melhora a estabilidade numérica
- Coloca as características em escalas semelhantes
- Facilita o uso de funções polinomiais e RBFs



CARACTERÍSTICAS POLINOMIAIS PARA DADOS 2D

- Para dados com duas dimensões, foram acrescentadas características polinomiais

```
x = Xs[:, 0:1]
```

```
y = Xs[:, 1:2]
```

```
polinomiais = torch.cat([
```

```
    x ** 2,
```

```
    x * y,
```

```
    y ** 2,
```

```
    x ** 3,
```

```
    (x ** 2) * y,
```

```
    x * (y ** 2),
```

```
    y ** 3
```

```
], dim=1)
```

- Características criadas:
 - x^2 , xy , y^2 , x^3 , x^2y , y^3
- Objetivo representar:
 - Curvas
 - Interações entre as variáveis
 - Fronteiras de decisão não lineares

FUNÇÕES DE BASE RADIAL – RBF

- As RBFs representam a proximidade dos pontos em relação a determinados centros
- A ideia é:
 - Se o ponto estiver próximo do centro, o valor da RBF será alto
 - Se estiver distante o valor será baixo

- **Cálculo da distância:**

```
diff = Xs[:, None, :] - self.centers[None, :, :]
```

```
d2 = (diff * diff).sum(2)
```

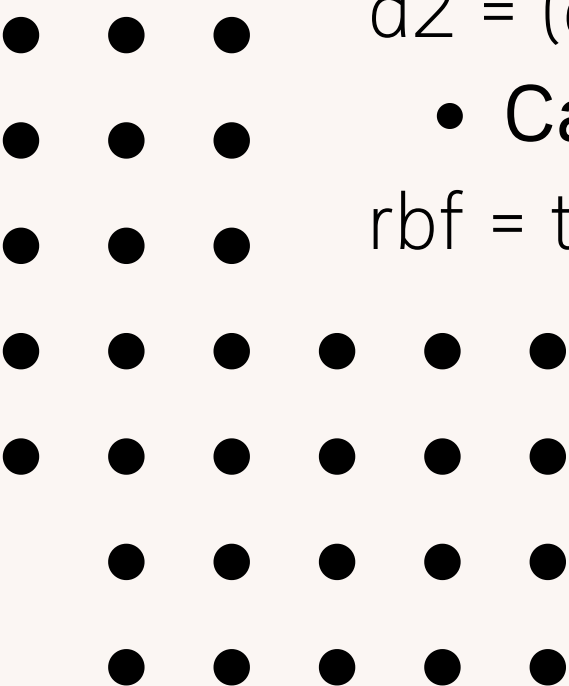
- **Cálculo da RBF:**

```
rbf = torch.exp(-self.gamma * d2)
```

Utilização

As RBFs ajudam principalmente nos conjuntos:

- duas_luas
- circulos
- espiral



ESCOLHA DOS CENTROS DAS RBFS

Os centros das RBFs são escolhidos a partir dos dados de treinamento
Foi utilizadas uma estratégia de seleção de pontos distantes entre si, chamada Farthest-Point Sampling

Ideia do método

1. Escolher um primeiro centro
2. Calcular a distância dos demais pontos aos centros escolhidos
3. Selecionar o ponto mais distante
4. Repetir até obter a quantidade desejada de centros

Vantagens

- Distribui os centros pelo espaço dos dados
- Evita concentrar todos os centros em uma mesma região
- Melhora a representação de padrões locais

TRATAMENTO PARA DADOS COM MAIS DIMENSÕES

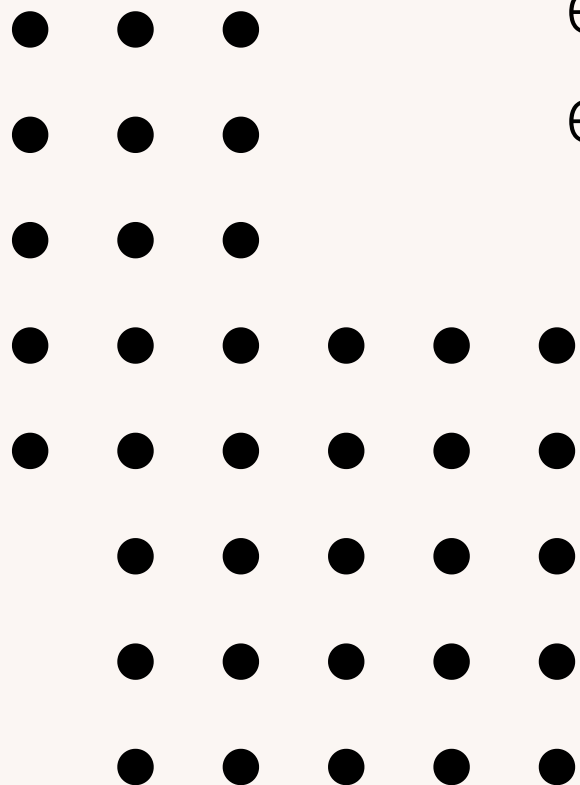
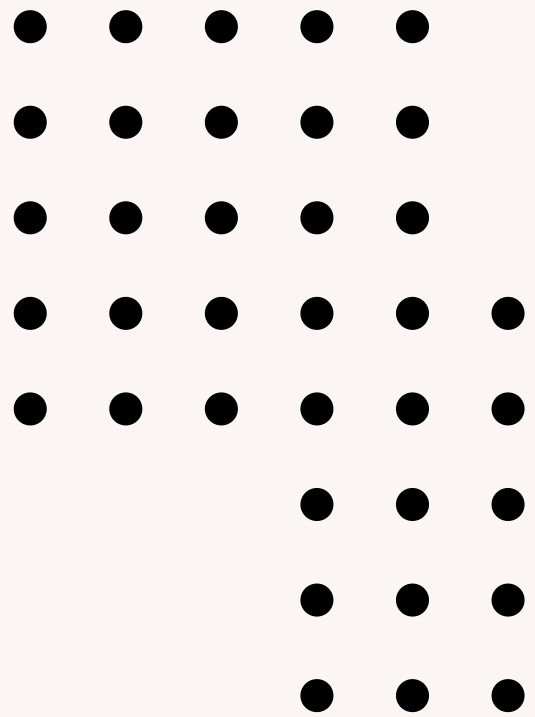
Para dados com mais de duas dimensões, foi acrescentada uma característica radial

```
r2 = (Xs ** 2).sum(dim=1, keepdim=True)
```

Essa característica representa:

$$r2 = x_1^2 + x_2^2 + \dots + x_d^2$$

Objetivo: a característica radial ajuda a representar a distância dos pontos em relação à origem. Isso é especialmente útil para o conjunto: esfera_10d



RANDOM FOURIER FEATURES – RFF

As dimensões restantes foram preenchidas com características obtidas por projeções aleatórias

```
proj = self._projetar(Xs)
```

```
rff = torch.cat([  
    torch.cos(proj),  
    torch.sin(proj)  
, dim=1)
```

A projeção utiliza uma matriz aleatória fixa:

```
self.W = torch.randn(self.d, k,  
generator=gerador)
```

- **Objetivo**

- Aumentar a capacidade de representação
- Criar características não lineares
- Utilizar melhor o limite de 64 dimensões

PROBLEMA DE DETERMINISMO

Durante os testes, a tarefa esfera_10d apresentou uma falha de determinismo.

O desafio compara:

`sub.phi(Xtr)`

com:

`sub.phi(Xtr[:20])`

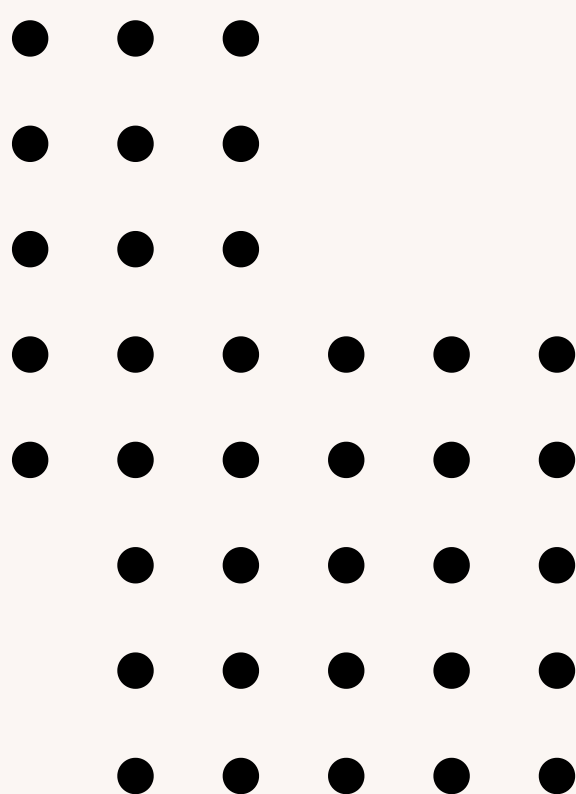
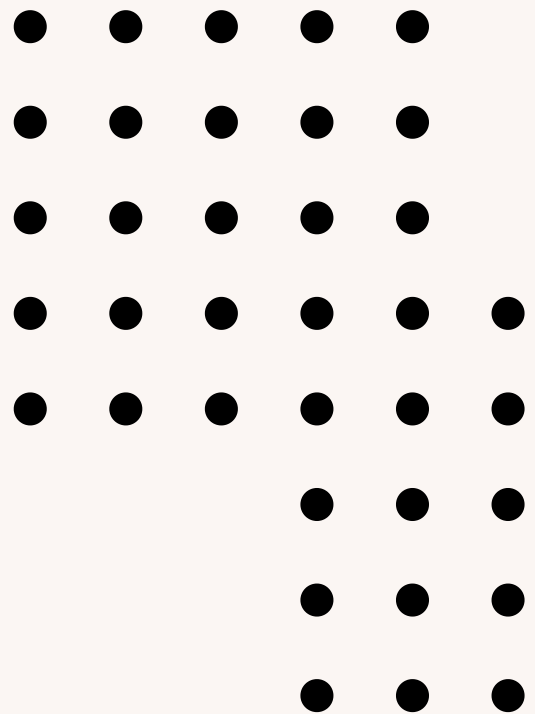
O problema estava relacionado à operação:

`Xs @ self.W`

Dependendo do tamanho do lote, podiam surgir pequenas diferenças numéricas

Consequência

A transformação era considerada não determinística e a tarefa recebia o resultado do baseline



CORREÇÃO APLICADA

A multiplicação matricial foi substituída por uma soma em ordem fixa

```
def _projetar(self, Xs):  
    P = Xs[:, 0:1] * self.W[0:1, :]  
  
    for j in range(1, self.d):  
        P = P + Xs[:, j:j + 1] * self.W[j:j + 1, :]  
  
    return P
```

- **Vantagem**

- A soma é realizada sempre na mesma ordem
- O resultado não depende do tamanho do lote
- A transformação passa a ser determinística
- O teste do desafio é atendido

RESULTADOS ESPERADOS

- Melhor representação dos dados não lineares
- Maior capacidade de classificação pelo Perceptron
- Melhor desempenho em conjuntos 2D
- Tratamento específico para esfera_10d
- Correção do teste de determinismo

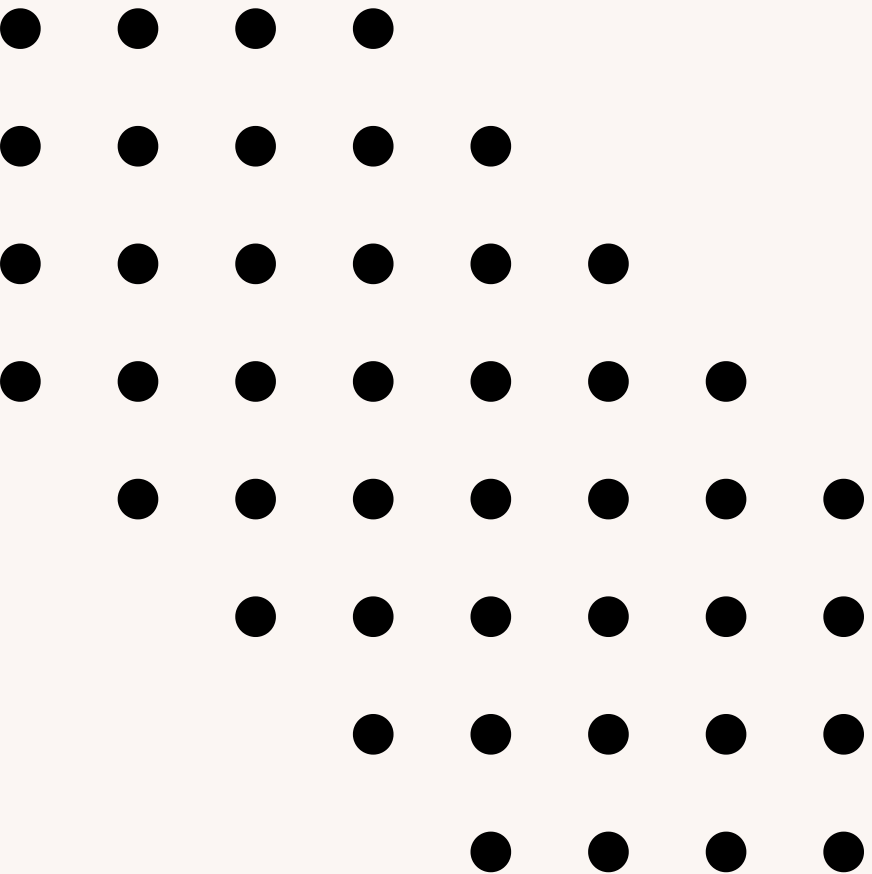
```
C:\Users\isabe\PycharmProjects\PythonProject\desafio11\venv\scripts\python.exe
tarefa      baseline  referência  você      s_t
xor          0.606     0.876       0.879     1.01
duas_luas    0.892     0.988       0.983     0.95
circulos     0.644     1.000       0.996     0.99
espiral      0.633     0.912       0.984     1.25
esfera_10d   0.508     0.872       0.984     1.25

ESCORE S = 104.7   (0 = baseline, 100 = referência, até 125 com bônus)

Process finished with exit code 0
```

CONCLUSÃO

- A transformação original foi ampliada para representar padrões mais complexos
- Foram utilizadas:
 - Padronização
 - Características polinomiais
 - RBFs
 - Característica radial
 - Random Fourier Features
- A correção numérica resolveu o problema de determinismo
- A solução tornou os dados mais adequados ao Perceptron
- Todo o processo respeitou o limite de 64 características



Obrigado!

